YENEPOYA INSTITUTE OF TECHNOLOGY

NBA-Accredited : B.E (CSE,IOT,BT)

(Affiliated to Visvesvaraya Technological University, Belagavi)

N.H. 13, Thodar, Moodbidri, Mangaluru 574225 (D.K), Karnataka

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(IOT,Cyber Security including Block Chain Technology)



LAB MANUAL

IOT COMMUNICATION PROTOCOLS

BCO701

[As per Choice Based Credit System (CBCS) scheme]

Vision

To empower students such that they will be technologically adept, innovation driven, self-motivated and responsible global citizens possessing human values by imparting quality technical education in the field of Computer Science.

Mission

1. To Facilitating and exposing the students to various learning opportunities through dedicated academic guidance and monitoring.
2. To Provide a learning ambience to encourage innovations, problem solving skills, leadership qualities, team-spirit, entrepreneurship skills and ethical responsibilities.

To Encourage faculty and students to actively participate in innovation, industry solutions,research and lifelong learning so that their contribution makes a substantial difference to the society.

Program Educational Objectives (PEO's):

- Graduates will be equipped to be employed in IT industries and be engaged in learning understanding and applying new ideas.
- Graduates through academic training will be able to take up higher studies and industry specific research programs.
- Graduates will be responsible computing professional with social obligations in their own area of interest.

Program Specific Outcomes (PSO's):

- Graduates will be able to use the knowledge and ability to write programs and integrate them with the hardware/software products in the domains of embedded systems, databases/data analytics, network/web systems and mobile products.
- Graduate will be able to use knowledge of information science in various domains to identify research gaps and hence to provide solution to new ideas and innovations.

Program Outcomes

1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis**: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long le

13. arning in the broadest context of technological change.

Contents:

Ex.No	Contents	Page no
Experiment 1	To study IoT Protocols	1-3
Experiment 2	Controlling the Light Emitting Diode (LED) using a Push Button	4-6
Experiment 3	Detection of Light using a Photoresistor (LDR) with Arduino	7-9
Experiment 4	Interfacing of Temperature Sensor (LM35) with Arduino	10-12
Experiment 5	Interfacing of Relay with Arduino	13-16
Experiment 6	HTTP Communication with Arduino (ESP32) — Send & Receive Data via HTTP Requests	17-18
Experiment 7	To develop an application that measures the room temperature and posts the temperature value on the cloud platform	19-22
Experiment 8	Temperature & Humidity Measurement Using DHT11 Sensor and Arduino	23-24
Experiment 9	Building an Intrusion Detection System using Arduino and Ultrasonic Sensor	25-27
Experiment 10	Direction control of DC Motor using Arduino	28-29
Experiment 11	Building Intrusion Detection System with Arduino and Ultrasonic Sensor	30-36
Experiment 12	Simple Sensor-Based Application Using Arduino – Lab Manual	37-38

Experiment 1: To study IoT Protocols

1. 6LoWPAN – Compact Version

Theory

6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks) enables IPv6 communication over IEEE 802.15.4 wireless networks. It uses header compression, packet fragmentation, and mesh routing to support constrained IoT devices with low power, low memory, and low bandwidth. It forms the foundation for modern low-power IoT connectivity standards.

Significance

- Enables IPv6 addressing for small battery-operated sensor nodes.
- Supports Smart Homes, Smart Cities, Industrial IoT.
- Forms the core of advanced IoT standards like Thread and Matter.
- Ensures scalable, long-range, low-energy communication in IoT ecosystems.

2. IPv4 / IPv6 – Compact Version

Theory

IPv4 uses 32-bit addressing, offering ~4.3 billion addresses, while IPv6 uses 128-bit addressing, offering virtually unlimited address space. IPv6 supports auto-configuration, simplified headers, and built-in security features—making it suitable for connecting billions of IoT devices globally.

Significance

- Essential for massive IoT expansion requiring unique global addresses.
- Mandatory for protocols like 6LoWPAN, CoAP, RPL, IoTivity.
- Improves efficiency, routing, and security compared to IPv4.
- Removes dependence on NAT, enabling direct device-to-device communication.

3. WiFi (802.11) – Compact Version

Theory

WiFi is a medium-range wireless LAN technology operating in 2.4 GHz and 5 GHz bands. It provides high data throughput and supports direct IP communication, making it suitable for IoT devices that require fast data transfer, internet connectivity, and cloud integration.

Significance

- Supports high-bandwidth IoT applications (camera modules, smart appliances).
- Widely used in home automation and industrial IoT systems.
- Provides direct internet access for cloud-based dashboards and services.
- Mature, stable technology with extensive hardware support (ESP32/ESP8266).

4. Bluetooth (BLE) – Compact Version

Theory

Bluetooth Low Energy (BLE) is an ultra-low-power wireless communication protocol designed for short-range data exchange. BLE uses advertising, pairing, and GATT profiles to efficiently communicate sensor data and control commands with minimal energy usage.

Significance

- Ideal for wearables, health devices, beacons, fitness trackers.
- Extremely low energy consumption allows long battery life.
- Supports personal area networking and mobile-based IoT interactions. □ Widely adopted in consumer electronics and healthcare IoT devices.

5. MQTT – Compact Version

Theory

MQTT (Message Queuing Telemetry Transport) is a lightweight, publish–subscribe messaging protocol optimized for low-bandwidth, high-latency, and unreliable networks. It uses a broker to manage

communication between publishers and subscribers, making it ideal for distributed IoT systems.

Significance

- Efficient for remote telemetry, sensor networks, and real-time monitoring.
- Backbone of major IoT cloud platforms (AWS IoT, Azure IoT, HiveMQ, ThingsBoard).
- Minimal overhead enables large-scale deployment of thousands of devices.
- Supports reliable delivery through QoS levels (0, 1, 2).

Resource

Cooja Contiki for 6LoWPAN and COAP Protocol https://sourceforge.net/projects/contiki/files/latest/download

Simulation video link - https://www.youtube.com/watch?v=eRQkgAKdiEk

IPV4 and IPV6 – Cisco Packet Tracer – Cisco Networking Academy
https://www.netacad.com/launch?id=ec0847b7-e6fc-4597-bc31-38ddd6b07a2f&tab=curriculum&view=8aac0445-6a6d-5cc2-9380-b72ca1cb6e05

Cisco Packet Tracer 9.0– Windows 64 bit - https://www.netacad.com/resources/labdownloads?courseLang=en-US

Packet tracer tutorials https://tutorials.ptnetacad.net/tutorials80.htm

Verify IPV4 and IPV6 Addressing - https://youtu.be/u9PG3uQ9BK0
WiFi – WiFi Scanning with wokwi - https://wokwi.com/projects/305569599398609473

Bluetooth – BLE Basics https://youtu.be/JSQhRyTKnW4?si=45-Sgf5HCpvl3otG

MQTT simulation from hive https://www.hivemq.com/demos/websocket-client/

MQTT Simulation video – .https://youtu.be/--4SETHIk7w?si=Mh9B-pbhnlffcM68 .

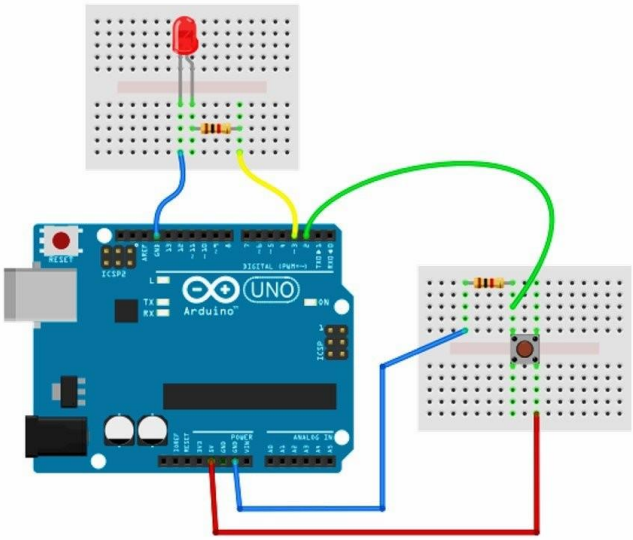
Experiment 2: Controlling the Light Emitting Diode (LED) using a Push Button

Aim:

To control the ON/OFF state of an LED using a push button as an input switch connected to an Arduino microcontroller.

Theory:

A **Light Emitting Diode (LED)** is a semiconductor device that emits light when current flows through it. The LED glows when forward biased and turns off when reverse biased. A **push button** acts as a **digital input device**, providing HIGH (1) or LOW (0) signals to the microcontroller. When the button is pressed, it completes the circuit allowing current to flow, which the Arduino reads as HIGH. Using a simple logic in the Arduino program, we can control the LED to turn ON when the button is pressed and OFF when released.
This experiment demonstrates **digital input-output interfacing**, which is a fundamental concept in embedded systems and automation.



Apparatus Required:

- Arduino Uno board
- Breadboard
- LED (any color)
- 220Ω resistor (for LED)
- Push button switch
- 10kΩ resistor (pull-down resistor)
-
- USB cable for programming

Circuit Connections:

1. Connect the **anode (long leg)** of the LED to **digital pin 13** of Arduino through a **1kΩ resistor**.
2. Connect the **cathode (short leg)** of the LED to **GND**.
3. Connect one terminal of the **push button** to **digital pin 2** of Arduino.
4. Connect the other terminal of the **push button** to **5V** supply.
5. Connect a **1kΩ resistor** between pin 2 and GND (as pull-down resistor).
6. Connect the Arduino board to the computer using the USB cable.

Procedure:

1. Open **Arduino IDE** on your computer.
2. Connect the **Arduino board** via USB cable and select the correct COM port.
3. Write or upload the following program: Program

```
int led = 13; int

button = 2; int

buttonState = 0;


void setup() {   pinMode(led,

OUTPUT);

pinMode(button, INPUT);

}


void loop() {   buttonState =

digitalRead(button);   if (buttonState ==

HIGH) {     digitalWrite(led, HIGH); //

LED ON

  } else {     digitalWrite(led, LOW);  //

LED OFF

  }
```

}

Observation / Output:

- When the **push button is pressed**, the LED **glows (ON)**.
- When the **button is released**, the LED **turns OFF**.
- This confirms successful interfacing of digital input and output devices.

Conclusion:

The LED can be effectively controlled by a push button using Arduino. The experiment demonstrates how **digital input (push button)** can be used to control **digital output (LED)**, forming the foundation for real-world embedded control systems like automation switches, counters, and signal indicators.

Experiment 3: Detection of Light using a Photoresistor (LDR) with Arduino

Aim:

To detect the presence and intensity of light using a **photoresistor (LDR)** and control an LED indicator through an Arduino based on the light condition.

Theory:

A **Light Dependent Resistor (LDR)** is a passive electronic component whose resistance varies inversely with the intensity of light falling on it.

- In **bright light**, the resistance of the LDR decreases, allowing more current to flow. In **darkness**,
- the resistance increases, reducing current flow.

By connecting the LDR in a **voltage divider circuit** with a fixed resistor, the varying voltage can be read by the **analog input pin** of an Arduino. The Arduino converts this analog voltage into a digital value (0–1023) and makes logical decisions — for example, turning an **LED ON** in darkness and **OFF** in bright light.

This principle is used in **automatic street lighting**, **light sensors**, and **solar tracking systems**.

Apparatus Required:

- Arduino Uno board
- **LDR (Light Dependent Resistor)**
- **10 kΩ resistor** (for voltage divider)
- **1 kΩ resistor** (for LED current limiting)
- LED
- Breadboard
- Jumper wires
- USB cable

Circuit Connections:

1. Connect one terminal of the **LDR** to **5 V**.
2. Connect the other terminal of the **LDR** to **Analog pin A0** and also to **GND** through a **10 kΩ resistor**.
3. Connect the **anode** of the LED to **Digital pin 13** via a **1 kΩ resistor**.
4. Connect the **cathode** of the LED to **GND**.
5. Connect the Arduino board to the computer via the **USB cable**.

Procedure:

1. Assemble the circuit on the breadboard as shown above.
2. Open **Arduino IDE** and connect the Arduino Uno to your computer.

3. Write and upload the following program
4. Open the **Serial Monitor** (Ctrl + Shift + M) to observe real-time readings.
5. Shine light or cover the LDR and note the LED behavior.

Program

```
int ldrPin = A0; int

ledPin = 13; int

ldrValue = 0;


void setup() {    pinMode(ledPin,

OUTPUT);

  Serial.begin(9600);

}


void loop() {    ldrValue =

analogRead(ldrPin);

  Serial.print("LDR Value: ");   Serial.println(ldrValue);


  if (ldrValue < 400) {         // Threshold for darkness     digitalWrite(ledPin,

HIGH); // LED ON

  } else {     digitalWrite(ledPin, LOW);  //

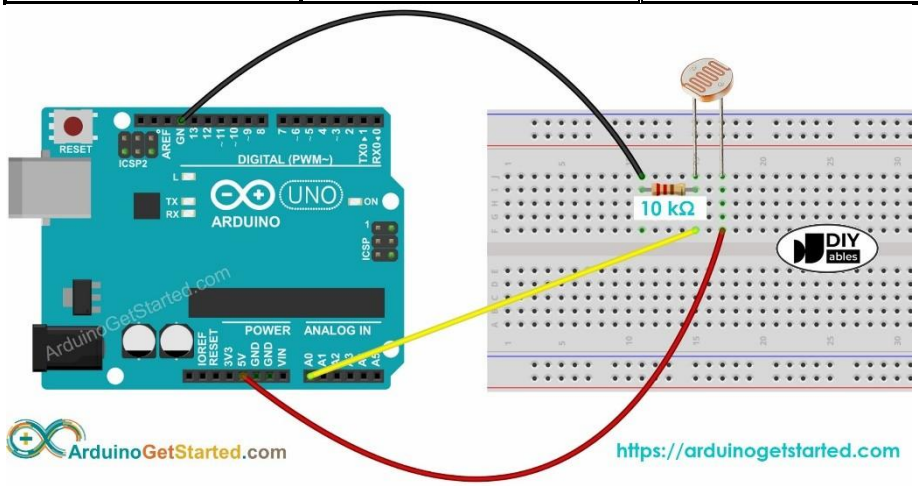
LED OFF

  }


  delay(500);

}
```

Light Condition	LDR Value (approx.)	LED Status
Bright light	700–900	OFF
Dim light	400–600	OFF/ON (threshold region)
Dark environment	0–300	ON



Conclusion:

The circuit successfully detects ambient light using a **photoresistor (LDR)** and responds by controlling an **LED**.
Using a **1 kΩ resistor** for the LED ensures safe operation while slightly reducing brightness — still adequate for indication.
This experiment illustrates the concept of **analog sensing**, **voltage division**, and **automated light control**, forming the foundation of smart lighting and IoT-based illumination systems.

Experiment 4: Interfacing of Temperature Sensor (LM35) with Arduino

Aim:

To measure the surrounding temperature using the **LM35 temperature sensor** and display the readings on the **Serial Monitor** or other output devices through Arduino.

Theory:

The **LM35** is a **precision integrated-circuit temperature sensor** whose output voltage is linearly proportional to the Celsius temperature. It provides an output of **10 mV per °C** of temperature. For example, at 25°C, the output voltage will be 250 mV.

The sensor operates between **–55°C to +150°C**, and since its output is analog, the voltage can be read using an **Arduino analog input pin (A0)**.

The Arduino's **Analog-to-Digital Converter (ADC)** converts the analog signal (0–5 V) to a digital value (0–1023). By applying a conversion formula, the temperature can be displayed in degrees Celsius.

This experiment demonstrates **sensor interfacing**, **analog data acquisition**, and **signal processing** with Arduino.

Apparatus Required:

- Arduino Uno board
- LM35 temperature sensor
- Breadboard
- Jumper wires
- USB cable for programming

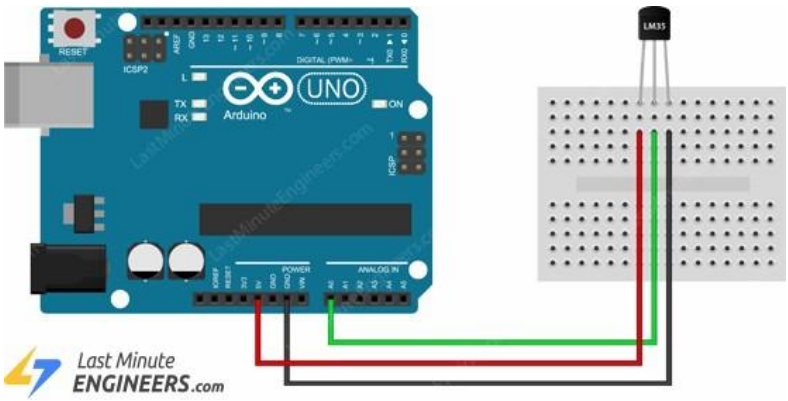
Pin Configuration of LM35:

- **Pin 1 (Vcc):** +5V supply
- **Pin 2 (Vout):** Output voltage proportional to temperature **Pin 3 (GND):** Ground

Circuit Connections:

1. Connect **Pin 1 (Vcc)** of LM35 to **5V** on Arduino.
2. Connect **Pin 3 (GND)** of LM35 to **GND** on Arduino.
3. Connect **Pin 2 (Vout)** of LM35 to **Analog input A0** on Arduino.
4. Connect the Arduino to the computer via USB cable.

Circuit diagram



Circuit diagram pin connections

+5V ---- LM35 (Vcc)

A0 ---- LM35 (Vout) GND

---- LM35 (GND)

Procedure:

1. Make the circuit connections as per the diagram above.
2. Open **Arduino IDE** and select the correct **board** and **COM port**.
3. Write and upload the following program: **Program**

```
int sensorPin = A0; float

sensorValue = 0; float

temperature = 0;


void setup() {

  Serial.begin(9600);

}


void loop() {    sensorValue = analogRead(sensorPin);   //

Convert ADC value to °C    temperature = (sensorValue *

5.0 * 100.0) / 1024.0;
```

```
  Serial.print("Temperature: ");

  Serial.print(temperature);

  Serial.println(" °C");   delay(1000);


}
```

Condition	Approx. Output Voltage (Vout)	Temperature (°C)
Room temperature	0.25–0.30 V	25–30 °C
Hand near sensor	0.35–0.40 V	35–40 °C
Warm surface contact	0.45–0.50 V	45–50 °C

Observation: The output voltage increases linearly with temperature rise.

Conclusion:

The **LM35 sensor** successfully measures the ambient temperature and outputs an analog voltage proportional to it. The Arduino reads this voltage and converts it into temperature readings in **degrees Celsius**.

This experiment demonstrates the **interfacing of an analog temperature sensor** and highlights the concept of **analog-to-digital conversion** and **sensor calibration** in embedded systems.

Experiment 5: Interfacing of Relay with Arduino

Aim:

To interface a **Relay Module** with an **Arduino Uno** to control high-voltage or high-current devices such as a bulb or fan through a low-voltage digital signal.

Theory:

A **relay** is an **electromagnetic switch** used to control the operation of high-power devices using lowpower control signals from a microcontroller. It consists of an **electromagnet (coil)**, a **movable armature**, and **switching contacts (COM, NO, NC)**.

When a small control signal is applied to the relay coil through Arduino, it gets magnetized and pulls the armature to switch between the **Normally Open (NO)** and **Normally Closed (NC)** terminals.

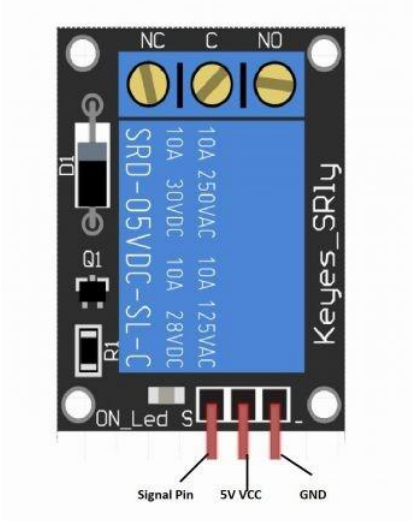
- When **digital output = HIGH**, the relay activates (switch closes).
- When **digital output = LOW**, the relay deactivates (switch opens).

The relay isolates the control circuit (Arduino) from the high-voltage circuit, ensuring safety and allowing **electromechanical control** of AC devices.

Apparatus Required:

- Arduino Uno board
- Relay module (5V SPDT or SRD-05VDC-SL-C)
- LED or small AC bulb (as load)
- 1 kΩ resistor (for LED)
- Breadboard
- Jumper wires
- USB cable

Pin	Function	Connection
VCC	Power Supply	+5V on Arduino
GND	Ground	GND on Arduino
IN	Control Signal	Digital Pin 7 (example)
COM	Common Terminal	Connected to load
NO	Normally Open	Connect to live wire of bulb
NC	Normally Closed	Not used (optional)



Circuit Connections:

1. Connect **VCC** of relay module to **5V** on Arduino.
2. Connect **GND** of relay module to **GND** on Arduino.
3. Connect **IN** pin of relay module to **Digital pin 7** of Arduino.
4. Connect one end of the **bulb** or **LED** to the **NO** terminal of the relay.
5. Connect the **COM** terminal of the relay to **5V (for LED)** or **AC live (for bulb)** as applicable.
6. For LED testing, connect its other leg to **GND** through a **1 kΩ resistor**.
7. Connect Arduino to the computer using the **USB cable**.

Circuit diagram connections – Pin connections

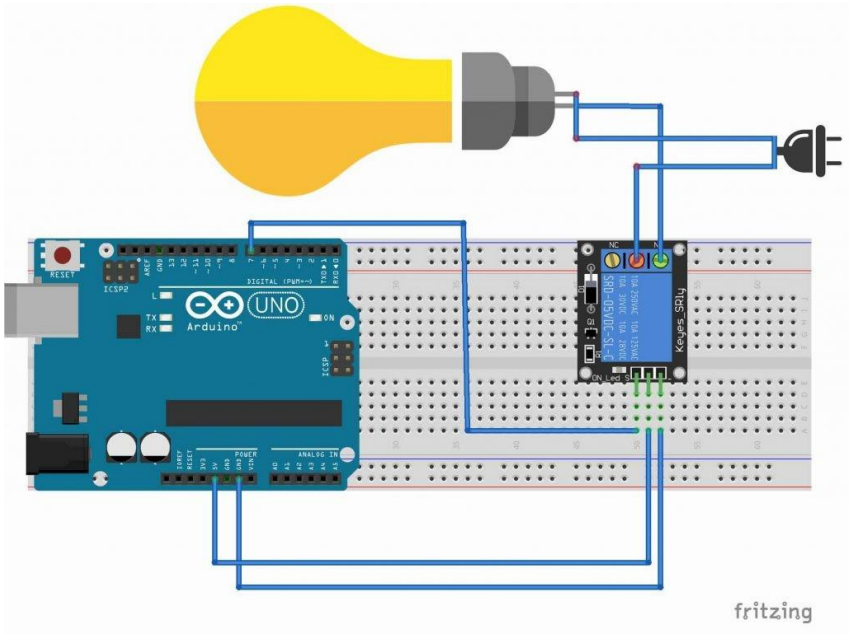
Arduino Pin 7 ----> IN (Relay)

5V ----> VCC (Relay)

GND ----> GND (Relay)

Relay COM ----> 5V

Relay NO ----> LED (with 1kΩ) ----> GND



Procedure:

1. Assemble the circuit as per the connections shown above.
2. Open **Arduino IDE** and select **Arduino Uno** and the correct **COM port**.
3. Enter and upload the following program
4. Observe the **clicking sound** of the relay as it switches ON and OFF.
5. The connected **LED** or **bulb** will turn ON and OFF accordingly.

Program

```
int relayPin = 7;


void setup() {   pinMode(relayPin,
OUTPUT);
}


void loop() {   digitalWrite(relayPin, HIGH);  //
Relay ON   delay(2000);                  // Wait 2
seconds   digitalWrite(relayPin, LOW);   //
Relay OFF   delay(2000);                  // Wait 2
seconds
}
```

Observation / Output:

- When the Arduino output signal is **HIGH**, the **relay activates**, closing the circuit and turning the **load ON**.
- When the signal is **LOW**, the **relay deactivates**, opening the circuit and turning the **load OFF**.

Conclusion:

The experiment successfully demonstrates the **interfacing of a relay** with Arduino to control external devices. The relay acts as a **switching interface** that allows the Arduino to control **high-power electrical loads** using **low-power control signals**, a principle widely used in **home automation, robotics, and industrial control systems**.

Experiment 6: HTTP Communication with Arduino (ESP32) — Send & Receive Data via HTTP Requests

Aim:

To send and receive simple data between ESP32 and a web server using HTTP requests.

Circuit:

No external circuit required — only **ESP32** and **Wi-Fi** connection.

Program

```
#include <WiFi.h> #include
<HTTPClient.h>

const char* ssid = "Your_WiFi_Name"; const char*
password = "Your_WiFi_Password";

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  Serial.print("Connecting to WiFi");   while
(WiFi.status() != WL_CONNECTED) {
delay(500);
    Serial.print(".");
  }
  Serial.println("\nConnected!");
}

void loop() {
  if (WiFi.status() == WL_CONNECTED) {




HTTPClient http;

    // --- HTTP POST Example ---
    String postData = "temperature=28.5&humidity=60";     http.begin("https://postman-echo.com/post"); //
test API endpoint     http.addHeader("Content-Type", "application/x-www-form-urlencoded");

    int postCode = http.POST(postData);     String
postResponse = http.getString();

    Serial.println("POST Response Code: " + String(postCode));
```

```
    Serial.println("Response: " + postResponse);     http.end();

    // --- HTTP GET Example ---
    http.begin("https://postman-echo.com/get?device=ESP32");     int
 getCode = http.GET();
    String getResponse = http.getString();
    Serial.println("GET Response Code: " + String(getCode));
    Serial.println("Response: " + getResponse);

    http.end();
  } else {
    Serial.println("WiFi Disconnected!");
  }

  delay(10000); // repeat every 10 seconds
}
```

Output example Connecting

to WiFi.....

Connected!

POST Response Code: 200

Response: { "form": { "temperature": "28.5", "humidity": "60" }, ... }

GET Response Code: 200

Response: { "args": { "device": "ESP32" }, ... }

Conclusion:

The program successfully demonstrates **bi-directional communication** between Arduino (ESP32) and a web service using **HTTP requests**, forming the basis for IoT applications like remote monitoring and control.

Experiment 7: To develop an application that measures the room temperature and posts the temperature value on the cloud platform Aim

To measure room temperature using a digital temperature sensor and **post the value to a cloud platform** over Wi-Fi using **HTTP**.

Theory

An **ESP32** has built-in Wi-Fi, allowing it to act as an **HTTP client**. A digital temperature sensor like **DHT11/DHT22** provides calibrated temperature data over a single GPIO pin. The ESP32 reads the sensor, formats the measurement, and **transmits** it to a cloud endpoint (e.g., **ThingSpeak**) using an HTTP **GET/POST** request. The server stores the data and renders it as charts for monitoring and analysis. This demonstrates end-to-end **sensor → microcontroller → network → cloud** data flow, fundamental to IoT systems.

Learning Outcomes

- Interface a **DHT11/DHT22** with ESP32.
- Connect ESP32 to Wi-Fi and **send HTTP requests**.
- Configure a simple **cloud channel** and visualize live data.

Apparatus Required

- ESP32 DevKit (NodeMCU-32S or similar)
- **DHT11** or **DHT22** sensor (3-pin module preferred)
- 1 × 10 kΩ resistor (only if using raw DHT sensor without a module)
- Breadboard and jumper wires
- USB cable (data)
- Computer with **Arduino IDE** (+ ESP32 boards installed)
- Cloud account (e.g., **ThingSpeak**) with a **Write API Key**

Alternative sensor: LM35 can be used with ESP32 ADC; however, DHT sensors are simpler (digital, no calibration code).

Cloud Setup (ThingSpeak example)

1. Sign in and create a **Channel**.
2. Enable **Field1** (rename as “Temperature (°C)”).
3. Note down your **Write API Key** (e.g., ABCD1234...).
4. The **update** endpoint is:
    - http://api.thingspeak.com/update?api_key=YOUR_KEY&field1=VALUE

Arduino Code (ESP32 + DHT, HTTP GET to ThingSpeak)

```
#include <WiFi.h>

#include "DHT.h"

#include <HTTPClient.h>
```

```
// -------- USER CONFIG -------const char* ssid     = "YOUR_SSID";
const char* password = "YOUR_PASSWORD"; const char*
writeApiKey = "YOUR_THINGSPEAK_WRITE_KEY";


// DHT setup

#define DHTPIN 4        // GPIO where DHT data pin is connected
#define DHTTYPE DHT11   // or DHT22
DHT dht(DHTPIN, DHTTYPE);


// Upload interval (ms) const unsigned long uploadInterval = 15000;



unsigned long lastUpload = 0;
void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");   while
(WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
delay(500);
  }
  Serial.print("\nConnected. IP: ");
  Serial.println(WiFi.localIP());
}
void setup() {
Serial.begin(115200);
dht.begin();   connectWiFi();
}


void loop() {   if (WiFi.status() != WL_CONNECTED)
connectWiFi();
```

```
  unsigned long now = millis();   if (now -
lastUpload >= uploadInterval) {
lastUpload = now;



   // Read sensor     float t =
dht.readTemperature(); // Celsius
   if (isnan(t)) {
     Serial.println("DHT read failed. Skipping this cycle.");
return;
   }
   Serial.print("Temperature (C): ");
   Serial.println(t, 1);

   // Build URL for GET
   String url = String("http://api.thingspeak.com/update?api_key=") +
writeApiKey + "&field1=" + String(t, 2);

   HTTPClient http;
http.begin(url);
 int code = http.GET();
if (code > 0) {
     String payload = http.getString();
     Serial.printf("HTTP %d, response: %s\n", code, payload.c_str());
   } else {
     Serial.printf("HTTP request failed: %s\n", http.errorToString(code).c_str());     }
   http.end();
  }
}
```

Notes:

- Keep the **upload interval ≥ 15 s** to respect ThingSpeak limits. **Procedure**

1. Assemble the circuit as per the connections.

2. In **Arduino IDE**, install **ESP32 board support** and **DHT sensor libraries** (*DHT Sensor Library by Adafruit* and *Adafruit Unified Sensor*).
3. Replace **YOUR_SSID**, **YOUR_PASSWORD**, and **YOUR_THINGSPEAK_WRITE_KEY** in the code.
4. Select **Board: ESP32 Dev Module**, choose the correct **Port**, and click **Upload**.
5. Open **Serial Monitor (115200 baud)** to confirm Wi-Fi connection, temperature readings, and HTTP status responses.
6. In **ThingSpeak**, view your **Channel dashboard** to observe the live temperature plot.

Observation / Expected Output

- **Serial Monitor:** Displays ESP32 IP address, temperature (°C) every ~15 s, and HTTP response code 200 with an entry ID.
- **ThingSpeak Dashboard:** Field 1 chart updates automatically with each new temperature reading.

Result / Conclusion

The ESP32-based system accurately measures room temperature using the DHT sensor and transmits data to the **ThingSpeak cloud** via HTTP. The readings are stored and visualized online, demonstrating a complete **IoT telemetry pipeline** from sensor acquisition to cloud analytics.

Experiment 8: Temperature & Humidity Measurement Using DHT11 Sensor and Arduino

1. Aim

To measure temperature and humidity using a DHT11 sensor with Arduino and display the values on the Serial Monitor.

2. Theory

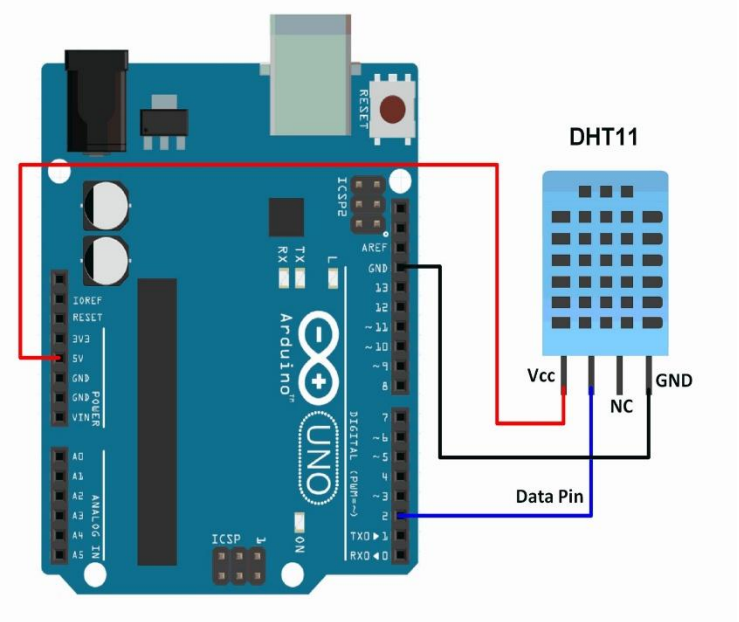
DHT11 is a digital sensor that measures temperature (°C) and relative humidity (%). It uses single-wire communication to send calibrated data to the Arduino. Arduino reads this data and shows it on the Serial Monitor.

3. Apparatus Required

1. Arduino Uno
2. DHT11 Sensor Module
3. Jumper wires
4. Breadboard
5. USB Cable
6. Arduino IDE with DHT library installed

4. Circuit Connections

DHT11 to Arduino Uno:
- VCC → 5V
- GND → GND
- DATA → Digital Pin 2 (with inbuilt pull-up or external 10k resistor to VCC)



5. Arduino Code

```
#include
"DHT.h"

#define DHTPIN 2
#define DHTTYPE DHT11 DHT

dht(DHTPIN, DHTTYPE);

void setup() {   Serial.begin(9600);
dht.begin();
}

void loop() {
  float h = dht.readHumidity();
  float t = dht.readTemperature(); // Celsius

  if (isnan(t) || isnan(h)) {
Serial.println("Sensor error");
delay(2000);     return;   }

  Serial.print("Temp: ");
  Serial.print(t);
  Serial.print(" °C  Humidity: ");
  Serial.print(h);
Serial.println(" %");
  delay(2000);
}
```

6. Observation

Note the values of temperature and humidity displayed on the Serial Monitor for different room conditions.

Table

Sl NO	Temperature °C	Humidity (%)	Remarks

7. Result

Temperature and humidity were successfully measured using the DHT11 sensor and displayed on the Serial Monitor through Arduino.

8. Conclusion

A simple sensor-based application was developed using Arduino and DHT11, demonstrating basic digital sensor interfacing and real-time environmental monitoring.

Experiment 9: Building an Intrusion Detection System using Arduino and Ultrasonic Sensor

Aim:

To design and develop an Intrusion Detection System that detects motion or unauthorized entry by using an Ultrasonic Sensor (HC-SR04) and triggers an alert through a buzzer or LED using an Arduino microcontroller.

Theory:

An ultrasonic sensor works on the principle of sound wave reflection (echo principle). It emits ultrasonic waves (typically at 40 kHz) from the TRIG pin. These waves reflect back upon hitting an object, and the ECHO pin receives them. The time taken for the echo to return is measured by the Arduino, which then calculates distance using the formula: Distance (cm) = (Time (μs) × 0.0343) / 2. When the measured distance becomes less than a set threshold (e.g., 20 cm), the system assumes an intruder is detected and activates an alarm (buzzer/LED). This forms a simple security system that can later be integrated with IoT platforms (Firebase, Blynk, etc.) for remote alerts.

Apparatus Required: -

Arduino Uno board

- Ultrasonic Sensor (HC-SR04)
- Buzzer or LED
- 220 Ω resistor (if using LED)
- Breadboard and Jumper wires
- USB cable

Circuit Connections:

Component	Arduino Pin	Description
VCC (HC-SR04)	5V	Power supply
GND (HC-SR04)	GND	Ground
TRIG	Digital Pin 9	Trigger pin
ECHO	Digital Pin 10	Echo signal input
Buzzer (+)	Digital Pin 7	Alarm output
Buzzer (–)	GND	Ground

Procedure:

1. Connect the ultrasonic sensor, buzzer, and power lines as per the circuit diagram.
2. Open Arduino IDE, connect Arduino Uno through USB, and select the correct port.
3. Copy and upload the following code.

Arduino Program:

```
#define trigPin 9
#define echoPin 10 #define
buzzer 7

long duration; int
distance; void
setup() {
  pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);   pinMode(buzzer,
OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(trigPin, LOW);
delayMicroseconds(2);   digitalWrite(trigPin,
HIGH);   delayMicroseconds(10);
digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH);   distance =
duration * 0.0343 / 2;

  Serial.print("Distance: ");
  Serial.print(distance);   Serial.println("
cm");

  if (distance < 20) {
    digitalWrite(buzzer, HIGH);
    Serial.println("⚠ Intruder Detected!");
  } else {
    digitalWrite(buzzer, LOW);
  }
  delay(500);
}
```

Observation / Output:

When no object is in range, the serial monitor shows distance > 20 cm and buzzer remains off. When an object or person comes within 20 cm, the buzzer activates, and '⚠ Intruder Detected!' appears on the Serial Monitor.

Result:

An Intrusion Detection System was successfully implemented using an ultrasonic sensor and Arduino Uno. The system accurately detects nearby movement and activates an alert system when an obstacle or person enters the restricted zone.

Conclusion:

This experiment demonstrates how Arduino and ultrasonic sensors can be used to design a basic security or motion detection system. It can be extended for smart home security, automated doors, or IoT-based surveillance.

Applications:

- Home or office security alarms
- Automatic gate/door control
- Parking assistance and obstacle detection systems
- Industrial safety systems

Experiment 10 : Direction Control of DC Motor using Arduino

In this experiment, we control both the speed and rotational direction of a DC motor using Arduino Uno. A potentiometer is used for speed control, and a tactile switch is used to reverse the motor's direction. The

L293D motor driver IC interfaces the DC motor with the Arduino, allowing safe control of direction and speed.

Theory:

The L293D motor driver IC is a dual H-Bridge that allows DC motors to be driven in both clockwise and counter clockwise directions. The Arduino generates PWM (Pulse Width Modulation) signals using the analogWrite() function to control motor speed. The direction of rotation is switched by toggling the logic levels at the L293D input pins via a tactile button using an external interrupt.

Apparatus Required: -

Arduino Uno

- L293D Motor Driver IC
- DC Motor
- 10kΩ Potentiometer
- Tactile Switch (Push Button)
- Breadboard and Jumper Wires
- External Power Supply (9V–12V)
- USB Cable

Circuit Connections:

Connection	Description
L293D Pin 1 (Enable 1)	Connect to 5V (Enable Motor Channel)
L293D Pin 2 (Input 1)	Arduino Pin 4
L293D Pin 7 (Input 2)	Arduino Pin 7
L293D Pins 3 & 6	Connected to DC Motor Terminals
L293D Pin 8	External +9V Motor Supply
L293D Pins 4,5,12,13	Ground
L293D Pin 16	5V from Arduino
Potentiometer Middle Pin	Arduino A1 (Speed Control Input)
Push Button One Side	Arduino Pin 2 (Interrupt Input)
Push Button Other Side	GND

Arduino Code: const int POT_input = A1; /* assign ADC Channel */ bool d1 = HIGH; bool d2 = LOW;

```
void setup() {
  pinMode(4, OUTPUT);  /* Motor control pin 1 */   pinMode(7,
OUTPUT);  /* Motor control pin 2 */   pinMode(3, OUTPUT);
/* PWM pin for Speed Control */   pinMode(2,
INPUT_PULLUP);  /* Interrupt pin for direction control */
  attachInterrupt(digitalPinToInterrupt(2), motor, FALLING);  /* Interrupt on falling edge */
}

void loop() {
int pwm_adc;
  pwm_adc = analogRead(POT_input); /* Input from Potentiometer for speed control */
digitalWrite(4, d1);   digitalWrite(7, d2);
  analogWrite(3, pwm_adc / 4);  /* PWM Output for speed control */
}

void motor(){
d1 = !d1;   d2
= !d2;
  _delay_ms(200);
}
```

Observation:

The motor speed varies according to the potentiometer position. Pressing the push button reverses the direction of rotation instantly. PWM control enables smooth variation of motor speed.

Result & Conclusion:

The speed and direction of a DC motor were successfully controlled using Arduino Uno, an L293D driver, a potentiometer, and a push button. This experiment demonstrates the use of PWM and interrupts for efficient motor control.

Experiment 11: Building Intrusion Detection System with Arduino and Ultrasonic Sensor

Aim

To measure distance using an ultrasonic sensor (HC-SR04) with ESP32 and send the distance values to Google Cloud via an HTTP endpoint.

Short Theory

- Ultrasonic sensor sends a sound pulse (Trig) and measures echo return time.
- Distance formula: Distance (cm) = (Time (µs) × 0.0343) / 2.
- ESP32 connects to Wi-Fi and sends distance values to a Google Cloud Function/HTTP endpoint using
- HTTP POST.

Apparatus

1. ESP32 Board
2. Ultrasonic Sensor HC-SR04
3. USB Cable
4. Jumper Wires + Breadboard
5. Wi-Fi Router
6. Arduino IDE
7. Google Cloud Function/HTTP endpoint URL

Circuit Connections

HC-SR04 to ESP32:
- VCC → 5V
- GND → GND
- TRIG → GPIO 5
- ECHO → GPIO 18

Google Cloud Setup – HTTP Endpoint

To measure distance using an ultrasonic sensor (HC-SR04) with ESP32 and send the distance values to Google Cloud via an HTTP endpoint, follow these steps to set up Google Cloud:

1. Create Google Cloud Project

- Go to console.cloud.google.com
- Click New Project
- Give project name → Create

2. Enable Required APIs

- Go to APIs & Services → Library
- Enable:
- Cloud Functions API
- Cloud Run API

- Cloud Build API

3. Create a Cloud Storage Bucket (Optional if using Cloud Functions only)

- Go to Cloud Storage
- Click Create Bucket
- Select Region
- Choose Uniform Access
- Create

4. Create an HTTP Endpoint using Cloud Functions

- Go to Cloud Functions → Create Function
- Choose:
- Environment → 1st gen
- Trigger → HTTP
- Authentication → Allow unauthenticated
- Runtime → Python / Node.js

Paste sample Python code: def hello_http(request): data = request.get_json() print(data) return 'Received'

- Click Deploy

5. Copy the Function URL

- After deployment, open the function
- Copy the Trigger URL
- This is your HTTP endpoint for ESP32

6. Set IAM Permissions (if needed)

If endpoint gives 403:

- Go to IAM → Add Member
- Add: allUsers

- Role: Cloud Functions Invoker

7. Test with POST Request

POST https://your-function-url.cloudfunctions.net/function-name

Content-Type: application/json

{"distance": 45}

Check Cloud Logging to confirm.

8. Use URL in ESP32 Code

ESP32 sends data like:

http.begin("https://your-function-url"); http.addHeader("Content-Type",

"application/json");

Arduino Code

```
/*
* ESP8266 NodeMCU + Ultrasonic Sensor + Buzzer + Google Cloud HTTP POST
* DUMMY VALUES FILLED
 */

#include <ESP8266WiFi.h>

// ---- WiFi & Cloud Config ---const char* ssid
= "DemoWiFi"; const char* password =
"DemoPassword123";

const char* host     = "demo-cloud-server.com";   // dummy domain
const int   httpPort = 80;                        // HTTP const char* path
= "/api/distance";          // dummy API path

// ---- Hardware Pins (NodeMCU) ----
#define TRIG_PIN    D1     // TRIG
#define ECHO_PIN    D2     // ECHO
```

```
#define BUZZER_PIN  D7     // Buzzer
#define DISTANCE_THRESHOLD 50   // cm

long  duration_us; float
distance_cm;

// ---------- Measure Distance ---------float
measureDistance() {
digitalWrite(TRIG_PIN, LOW);
delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
delayMicroseconds(10);   digitalWrite(TRIG_PIN,
LOW);

  duration_us = pulseIn(ECHO_PIN, HIGH, 30000);

  if (duration_us == 0) return -1;

  return (duration_us * 0.0343) / 2.0;
}

// ---------- Send Data to Google Cloud ---------void
sendToGoogle(float d) {
  WiFiClient client;

  Serial.print("Connecting to ");   Serial.println(host);

  if (!client.connect(host, httpPort)) {
Serial.println("Host connection FAILED");     return;
  }

  String payload = "{\"distance_cm\":" + String(d, 2) + "}";   String request =
```

```
    String("POST ") + path + " HTTP/1.1\r\n" +
    "Host: " + host + "\r\n" +
    "Content-Type: application/json\r\n" +
    "Content-Length: " + payload.length() + "\r\n" +
    "Connection: close\r\n\r\n" +     payload;

  Serial.println("------ HTTP REQUEST ------");
  Serial.println(request);
  Serial.println("--------------------------");

  client.print(request);

  while (client.connected() || client.available()) {     if
(client.available()) {
      String line = client.readStringUntil('\n');
      Serial.println(line);
    }
  }

  client.stop();
}

// ---------- Setup ---------void
setup() {
Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);   pinMode(ECHO_PIN,
INPUT);   pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);   WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
```

```
  while (WiFi.status() != WL_CONNECTED) {
 delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.print("Connected! IP: ");
  Serial.println(WiFi.localIP());
}

// ---------- Loop ---------void loop() {
distance_cm = measureDistance();

  Serial.print("Distance: ");
  Serial.print(distance_cm);   Serial.println("
cm");

  if (distance_cm > 0 && distance_cm < DISTANCE_THRESHOLD)
digitalWrite(BUZZER_PIN, HIGH);   else     digitalWrite(BUZZER_PIN,
LOW);

  sendToGoogle(distance_cm);

  delay(10000);
}
```

Observation

Record Serial Monitor readings and confirm values arriving in Google Cloud logs. Table:

Sl.No	Actual	Measured (serial monitor)	Seen on cloud

Result

Distance values were successfully measured and uploaded to Google Cloud endpoint.

Conclusion

A complete IoT system was implemented using ESP32 and HC-SR04 with Google Cloud integration.

Experiment 12: Simple Sensor-Based Application Using Arduino – Lab Manual

Aim

To develop a simple sensor-based application using an Arduino Uno and an LDR (Light Dependent Resistor) to measure light intensity and display the output on the Serial Monitor.

Short Theory LDR

Sensor:

- Light Dependent Resistor changes resistance based on light intensity.
- More light → lower resistance; less light → higher resistance.

Arduino Uno:

- ATmega328P-based microcontroller board.
- Reads analog values (0–1023) using its 10-bit ADC.

Apparatus Required

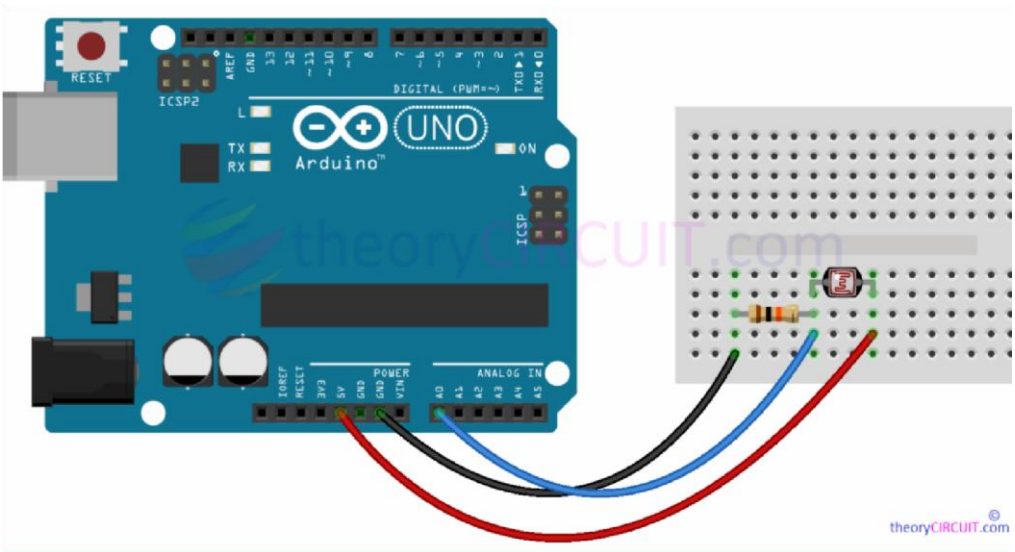
1. Arduino Uno
2. LDR (Light Dependent Resistor)
3. 10kΩ resistor (for voltage divider)
4. Breadboard
5. Jumper wires
6. USB Cable (Type-B)
7. Arduino IDE

Circuit Connections

Voltage Divider Circuit (LDR + 10k resistor):

- One end of LDR → 5V
- Other end of LDR → A0 (Analog Pin) -
  Same point → 10k resistor → GND

5V------LDR ---- A0 ---- 10kΩ ---- GND



Arduino Code

```
int sensorPin = A0;
int sensorValue = 0;


void setup() {
  Serial.begin(9600);
}

void loop() {    sensorValue = analogRead(sensorPin);

  Serial.print("Light Intensity Value: ");   Serial.println(sensorValue);

  delay(1000);
}
```

Observation Table

Sl.No	Light Condition	Sensor Value (0-1023)	Remarks
1	Bright Light		
2	Dim Light		
3	Darkness		

Result

The Arduino Uno successfully measured light intensity using an LDR and displayed the analog readings on the Serial Monitor.

Conclusion

This experiment demonstrates basic sensor interfacing with Arduino and real-time analog data acquisition using the LDR sensor.